Common setup (run once)

```
pip install nltk
```

```
import nltk

# Download the NLTK datasets/models needed by the lab programs.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('brown')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('universal_tagset')
nltk.download('conll2000')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
```

Run each program in Jupyter Notebook, Google Colab, or VS Code with Python installed. Some programs download corpora on first run and need an internet connection.

10. Find POS tags of words in a sentence using Viterbi decoding

Aim: To use an HMM and the Viterbi algorithm to identify the most probable sequence of POS tags for a sentence.

The Viterbi algorithm finds the tag sequence that maximizes the joint probability of the observed words and hidden tags.

```
import nltk
from nltk.corpus import brown
from nltk.tag import hmm

# Download the tagged training corpus.
nltk.download('brown')
nltk.download('universal_tagset')

# Use the first 500 Brown Corpus sentences for training.
tagged_sentences = brown.tagged_sents(
    categories='news',
    tagset='universal'
)[:500]

# Create an HMM trainer.
trainer = hmm.HiddenMarkovModelTrainer()

# Train the HMM using the tagged sentences.
tagger = trainer.train_supervised(tagged_sentences)

# Define the sentence to tag.
sentence = "the company announced new products".split()

# The HMM tagger uses Viterbi decoding to infer tags.
tagged_words = tagger.tag(sentence)

print("Input sentence:", sentence)
print("POS tags using Viterbi:")

for word, tag in tagged_words:
    print(word, "->", tag)
```

Sample output

```
Input sentence: ['the', 'company', 'announced', 'new', 'products']
POS tags using Viterbi:
the -> DET
company -> NOUN
announced -> VERB
new -> ADJ
products -> NOUN
```

Explanation:

1. Train an HMM from word-tagged sentences.
2. Supply the untagged input sentence.
3. Viterbi decoding searches for the most probable sequence of hidden POS tags.
4. The program prints each word with its predicted tag.

The sample tags are illustrative; the actual output depends on the trained model. Since the training corpus is small, accuracy may be limited. Unseen words can also cause issues in a basic HMM.